FINAL-YEAR PROJECT · ALIGNED WITH THE OWASP AI MODEL WATERMARKING PROJECT (NOT OFFICIAL OWASP WORK)

Sigil.

Sigil hides a secret mark inside a neural network's weights. If someone copies the model, the owner can prove it's theirs with a zero-knowledge proof that reveals nothing about the mark.

`64-bit mark in one layer` `accuracy 98.69% → 98.66%` `proof made in ~4 s` `checked in ~0.4 s`

HOW TO USE THIS GUIDE

Four parts, four presenters, about 17 minutes

The guide is split into four parts, one per team member, in the order you present them. Each part starts on its own page and says what to click, what to say, the numbers to quote, a question to expect and the line that hands over to the next person. Set-up and troubleshooting are on the last page.

PART	PRESENTER	TOPIC	ON SCREEN	TIME
1	**Arfat Shaikh**	The problem, the idea, the app	Home, Overview	~4 min
2	**Mohamed Ansari**	Hiding the mark, attacking it	Demos A, B, Model under test	~4 min
3	**Sami Daire**	Forged proofs, and our fix	Demos C–F, Verify ownership	~5 min
4	**Yash Bandiwadekar**	Behind the screen, real use, limits	Console, dispute, Overview	~4 min

THE WHOLE SYSTEM IN FIVE STEPS

Everyone should know this part

1. **Create an identity.** The owner makes an Ed25519 key pair and a secret random seed. Both stay encrypted on the owner's machine.
2. **Embed the mark.** A secret key and a 64-bit signature are derived from the seed and the owner's key, then trained into one layer of the model.
3. **Register.** Before release, the owner publishes a signed record: a commitment (hash) of the seed, the settings and the model file's SHA-256. A registry timestamps it.
4. **Open a claim.** Someone who suspects a copy opens a claim on that file, with a fresh random number (a nonce) so an old proof can't be reused.
5. **Prove and verify.** The owner proves in zero knowledge that the committed mark is in the suspect file, with at most 8 of 64 bits wrong. The verifier recomputes the file's fingerprint and checks the signature, the timestamp, the nonce and the proof.

BEFORE YOU START

Two minutes of checks

1. Double-click `Start-Demo-UI.cmd` and open `http://127.0.0.1:8765`. No "Simulate mode" banner means every run is real; "No ML stack" in the sidebar means use replay mode (last page).
2. Press **Clear results** so every page is empty: everything that appears next is computed in front of the audience.
3. Keep `python demo\server.py --simulate --port 8766` ready in a second terminal as a backup.

1 The problem, the idea and the app

Arfat Shaikh · ~4 min · owns the web app, SDK packaging and the command line, design

ON SCREEN

1. Open the **Home** page: the pitch, how it works, the team.
2. Press **Open dashboard** to reach the **Overview**. Point out the sidebar (Overview, Embed watermark, Verify ownership, Robustness lab, research demos A–F), the console at the bottom and the **Model under test** bar at the top.
3. Press **Run all live**. It fills demos A–E in about 30 seconds while you talk; the console streams every line as it happens.

WHAT TO SAY

- **The problem.** A trained model costs data, compute and time, and it is easy to copy. If someone republishes your model as theirs, you need proof that it is yours, without handing over your secret.
- **The idea has three pieces.** First, a hidden mark: 64 bits written into one layer's weights during training. Second, a commitment: before release, the owner publishes a hash of their secret and a registry timestamps it. Third, a zero-knowledge proof: the owner proves the mark is in a suspect file without revealing the secret.
- **What you are looking at.** A local web app. Every button runs real Python on this laptop, one job at a time. Nothing is sent to the cloud.
- **The six demos tell one story.** A embeds the mark, B attacks it, C shows that older ownership proofs can be forged, D shows how committing to a seed stops that, E shows that zero knowledge alone doesn't, and F is our complete fix with a real zero-knowledge proof.
- **For developers.** The same steps come as a Python SDK and a `sigil` command that work with PyTorch (.pt), Keras (.keras, .h5) and ONNX models.

NUMBERS TO QUOTE

`64-bit mark` `1 layer: conv2, 288 averaged weights` `accuracy 98.69% → 98.66%` `proof ~4 s` `check ~0.4 s` `3 model formats`

> **"Is this live or a recording?"**
>
> Live. There is no Simulate banner, and we cleared every page before starting, so everything on screen was computed just now. Replay mode exists only as a backup.

HAND-OFF "The run has finished. Mohamed will show how the mark gets into the model, and what happens when a thief attacks it."

2 Hiding the mark, and trying to remove it

Mohamed Ansari · ~4 min · owns embedding (PyTorch, Keras, ONNX), the attacks, the acceptance threshold

ON SCREEN

1. Open demo **A · Commit & embed** (or the **Embed watermark** page).
2. Open the **Robustness lab** (demo B): the attack table and the pruning sweep. Optionally press **Run attacks live** (about 10 s).
3. In the **Model under test** bar, test *pruned 90%*, then *channels permuted*, then *clean.pt* (4–15 s each).

WHAT TO SAY

- **How the mark is written.** Take the layer conv2 and average its weights into 288 numbers. Multiply them by a secret 64 × 288 key and read the sign of each result as one bit. During training, an extra loss pushes those 64 bits towards the owner's signature. The model still does its job: accuracy goes from 98.69% to 98.66%.
- **How it is read.** Count the wrong bits. The owner's model has 0 of 64 wrong. A model that never had the mark gets about half wrong, like coin flips.
- **The acceptance rule.** At most 8 of 64 bits may be wrong. We tested it on 30 models trained without a mark and none passed. For a random model the chance is about 3 in 10 billion.
- **The attacks.** Pruning up to 95% of the weights, 8-bit and 4-bit quantisation, fine-tuning, re-training, and a thief writing their own mark on top all leave the owner's mark readable. Permutation (reordering a layer's channels) scrambles it, but the owner can realign the model and read it again.
- **The live test.** *Pruned 90%* has 2% accuracy, a useless model, yet ownership is still proven. *Channels permuted*: no proof. *clean.pt*: no proof, as it should be.

NUMBERS TO QUOTE (DEMO B)

ATTACK ON THE STOLEN MODEL	ACCURACY	BITS WRONG	MARK FOUND
None (stolen as-is)	98.66%	0 / 64	yes
Prune 90% of the weights	9.41%	1 / 64	yes
4-bit quantisation	98.66%	0 / 64	yes
Fine-tune all layers, 5 epochs	98.84%	0 / 64	yes
Thief overwrites with their own mark	98.62%	3 / 64	yes
Channel permutation	98.66%	28 / 64	no
Permutation, then owner realigns	98.66%	0 / 64	yes

"So can a thief remove it?"

Not with these attacks without ruining the model, except permutation, which realignment undoes. Training a brand-new model from this model's answers (model extraction) does remove it. We list that as a limit.

HAND-OFF "Reading the bits shows the mark is there. Sami will show why that alone doesn't prove who owns the model."

3 Why proofs can be forged, and our fix

Sami Daire · ~5 min · owns the protocol, the zero-knowledge circuit, the verifier, integration

ON SCREEN

1. First press **Run demo F live** (about 80 s). Explain C, D and E while it runs.
2. Walk through demos **C · Ownership forgery**, **D · Committed seed** and **E · Zero-knowledge proofs**. The ← → keys move between demos.
3. When F finishes, open **Verify ownership**: the claim, the proof, the 12 checks and the verdict. Press each **Try a misuse** button (about 1 s each), then show the **On-chain** view.

WHAT TO SAY

- **C, the problem.** In earlier schemes the prover picks the key at proof time. A thief can fit a key so the owner's model reads "MALLORY!" with 0 bits wrong, and the same trick "finds" a mark in models that were never marked. The fake key's values have the same distribution as a real key's.
- **E, zero knowledge alone doesn't help.** With the EZKL tool, the thief's forged proof verifies under the owner's own verification key: the two keys are identical. A valid proof says nothing about who owns the model.
- **D, the fix is to commit first.** The owner publishes a hash of a secret seed at registration, and the key and signature are derived from that seed. A thief would have to find a seed that works: 0 of 20,000 tries came close (the best had 16 of 64 bits wrong). At 64 bits that is about 2^{32} tries, around 114 hours on one CPU core for an even chance. At 256 bits it is about 2^{120}.
- **F, the complete system.** A Circom circuit re-derives the key and signature from the committed seed inside the proof, allows at most 8 wrong bits, and is tied to this exact file and this claim. The verifier recomputes the file's fingerprint itself. A thief with their own key has 28 of 64 bits wrong, so no proof is possible.
- **The misuse buttons.** Each one changes one thing and the verifier rejects it with its own reason (table below). The same proof also verifies in a smart contract on a local blockchain.

NUMBERS TO QUOTE

`proof made in 3.7 s` `proof 879 bytes (EZKL: ~61 KB)` `12 of 12 checks pass` `thief: 28 / 64 wrong` `on-chain check: 243,225 gas`

MISUSE	VERIFIER'S REASON
Replay the proof for another claim	proof was made for a different claim (nonce mismatch)
Check it against a different model	proof is about a different model (w_hash mismatch)
Change some proof bytes	the zk-SNARK does not verify
Receipt from an untrusted registry	receipt is not from a trusted registry

> **"What does the verifier actually learn?"**
>
> Only that the registered owner's committed mark is in this exact file, with at most 8 of 64 bits wrong. Not the seed, not the key, not the signature.

HAND-OFF "Yash will show what runs behind the screen, who wins when two people claim the same model, and how this would be used for real."

4 Behind the screen, real-world use, and limits

Yash Bandiwadekar · ~4 min · owns the demo server and the FastAPI + PostgreSQL backend

ON SCREEN

1. Start any run (for example **Test this model**) and point at the console: each line appears as the script prints it. Show **Cancel run**.
2. Open demo **F** and scroll to the **overwrite dispute**: owner and thief, each with a registration time.
3. Finish on the **Overview**: the headline numbers and the audit log.

WHAT TO SAY

- **What happens when you press Run.** The page asks the local server to start one job. The server runs the matching Python script on this laptop and streams its output to the console; when it ends, the page reloads the results. Only one job runs at a time, each step has a time limit, and Cancel stops it cleanly.
- **Security.** The server only accepts connections from this machine and refuses requests from other websites. The seed and the private key stay encrypted and never appear in results or logs; a test scans every saved file for them.
- **The dispute.** A thief who writes their own mark on top can also make a valid proof, because both marks are now in the model (owner 1 bit wrong, thief 0). The registry timestamp decides: the owner registered at 21:43, the thief at 22:23, so the owner wins. That's why you register before you release.
- **For real use.** A FastAPI + PostgreSQL backend adds accounts with owner, verifier and admin roles, model uploads with size and type checks, registry receipts, verification history, an audit log where editing any row is detected, ownership certificates (PDF and signed JSON), and monitoring that raises an alert when a deployed model file changes. A verifier can also run `sigil report` to get the verdict as a written report.
- **Limits, said before anyone asks.** Training a new model from this model's answers removes the mark (in our tests the trigger-set mark too). Permutation hides it until the owner realigns. 64 bits is the demo size: a seed search is about 2^{32} tries, which a GPU can do, so the secure setting is 256 bits (proof in about 12 s). Groth16 needs a trusted setup; we use a public one plus our own contribution.

> **"What if the thief registers first?"**
>
> The earliest registration wins, so the owner registers before releasing the model. The registry signs every timestamp, so a receipt can't be back-dated.

CLOSE "Sigil writes a mark into the model, commits to it before release, and proves ownership in zero knowledge in about four seconds, without ever revealing the secret. Thank you."

LAST PAGE

Set up and run the project

Run every command from the project folder (the one with DEPLOY.md). Everything runs locally; nothing is sent to a cloud service.

Option 1: replay mode · no install · about 1 min · needs Python 3.10+

Run buttons replay recorded real runs. Use it as the backup during a presentation.

```
# or double-click Start-Demo-UI-Simulate.cmd
python demo\server.py --simulate
```

Option 2: live mode · 5–15 min setup · needs Python 3.10+ and Node.js 20.19+

1. **Install** PyTorch, the SDK, the circuit tools and the proving keys into `.venv`:

   ```
   .\setup.ps1 --demo              # macOS / Linux: ./setup.sh --demo
   ```

2. **Check** everything: it runs the tests, starts the app and makes a real proof.

   ```
   .venv\Scripts\python scripts\verify_deploy.py
   ```

   It must end with: ALL CHECKS PASSED

3. **Start** by double-clicking `Start-Demo-UI.cmd`, or run:

   ```
   .venv\Scripts\python demo\server.py --open
   ```

4. **Open** `http://127.0.0.1:8765` and press **Open dashboard**.

Don't run live: Retrain (it replaces the owner key) and E3 (it needs about 32 GB of RAM).

For developers: the command line

```
# owner
sigil keygen                                    # owner key, encrypted
sigil seed --bits 64 --layer conv2              # secret seed + settings
sigil embed model.pt --arch mymodels:Net --data train.npz
sigil register model.wm.pt --registry-key secrets/registry.key
# verifier opens a claim, owner proves, verifier checks and gets a report
sigil claim registration.json --suspect suspect.pt
sigil prove suspect.pt --claim claim.json --out bundle
sigil report bundle --suspect suspect.pt --registration registration.json --receipt
registration.receipt.json --trusted-registry <registry key>
```

IF YOU SEE	DO THIS
"Can't reach the demo server"	Start the server. If port 8765 is taken, add `--port 8770`.
"No ML stack: replay only"	Start with the `.venv` Python (Option 2), or present in replay mode.
Demo F says the circuit isn't set up	Re-run `setup.ps1 --demo`; it installs the proving keys.
Antivirus blocks `circom.exe`	It's the official circom 2.2.3 release. Allow it, or show demo F in replay mode.

More detail in the repository: `DEMO.md` (presenter runbook), `Sigil-User-Guide.pdf` (illustrated guide), `docs/protocol.md` (the protocol) and `OWASP AI Model Watermarking.pptx` (slides).